

An Empirical Study on Leveraging Images in Automated Bug Report Reproduction

Dingbang Wang
University of Connecticut
USA
dingbang.wang@uconn.edu

Zhaoxu Zhang
University of Southern California
USA
zhaoxuzh@usc.edu

Sidong Feng
Monash University
Australia
sidong.feng@monash.edu

William G. J. Halfond
University of Southern California
USA
halfond@usc.edu

Tingting Yu
University of Connecticut
USA
tingting.yu@uconn.edu

Abstract—Automated bug reproduction is a challenging task, with existing tools typically relying on textual steps-to-reproduce, videos, or crash logs in bug reports as input. However, images provided in bug reports have been overlooked. To address this gap, this paper presents an empirical study investigating the necessity of including images as part of the input in automated bug reproduction. We examined the characteristics and patterns of images in bug reports, focusing on (1) the distribution and types of images (e.g., UI screenshots), (2) documentation patterns associated with images (e.g., accompanying text, annotations), and (3) the functional roles they served, particularly their contribution to reproducing bugs. Furthermore, we analyzed the impact of images on the performance of existing tools, identifying the reasons behind their influence and the ways in which they can be leveraged to improve bug reproduction. Our findings reveal several key insights that demonstrate the importance of images in supporting automated bug reproduction. Specifically, we identified six distinct functional roles that images serve in bug reports, each exhibiting unique patterns and specific contributions to the bug reproduction process. This study offers new insights into tool advancement and suggests promising directions for future research.

Index Terms—Android, Bug report, Empirical study

I. INTRODUCTION

In the mobile app marketplace, debugging and problem resolution are critical. Research shows that 88% of app users are likely to abandon an app if they encounter persistent issues, emphasizing the importance of prompt resolution to retain users [1]. One significant challenge developers face is reproducing bugs reported by users, who often provide insufficient information, such as the exact sequence of their interactions [2]–[5]. To tackle this issue, the software engineering community is increasingly focused on automating the bug reproduction process [6]–[14].

Bug reproduction relies on the information provided in bug reports to replicate reported issues. From a functional perspective, a bug report typically includes essential components such as Steps to Reproduce (S2R), Expected Behavior (EB), and Observed Behavior (OB). In terms of media types, it

MSR 2025, Ottawa, Canada, April 28-29, 2025

TABLE I
SUMMARY OF RECENT ANDROID BUG REPRODUCTION WORKS

Tool Name	Venue	Input	Open Source	Dataset Size
Yakusu [6]	ISSTA' 18	S2Rs	✓	60
* ReCDroid [7]	ICSE' 19	S2Rs	✓	51
GIFdroid [14]	ICSE' 22	GIF \ Videos	✓	61
* ReproBot [9]	ISSTA' 23	S2Rs	✓	77
ScopeDroid [12]	ICSE' 23	S2Rs	×	102
* AdbGPT [10]	ICSE' 24	S2Rs	✓	48
CrashTranslator [13]	ICSE' 24	Error Log	✓	75
Roam [8]	FSE' 24	S2Rs	✓	72
* ReBL [11]	ISSTA' 24	Whole bug report	✓	96

¹ * indicates the tool is selected as baseline for this study.

may feature text descriptions, images, and videos/GIFs. In this paper, *images* refer to UI page *screenshots* or any *static visual information* provided by the user, which may be beneficial for bug reproduction process.

Table I summarizes the recent state-of-the-art automated bug reproduction tools, highlighting the specific bug report information they utilize and the main techniques employed in their approaches. Textual Steps to Reproduce (S2Rs) stand out as the most commonly used information employed by the majority of the works [6]–[10], as they provide the most straightforward instructions needed to trigger the bug. However, other efforts [11], [13], [14] have shown that information beyond textual S2Rs is critical for bug reproduction, yielding promising results. For instance, Feng et al. [14]–[16] employ videos and GIFs in bug reports for bug reproduction, while Huang et al. [13] utilize error log. Similarly, Wang et al. [12] leverages the entire textual bug report, including the title and other elements, demonstrating that each piece of information in bug reports can potentially supplement the S2Rs and improve the performance of automated bug reproduction.

However, none of these approaches consider the images provided in bug reports, and the potential for leveraging these images to assist in automated bug reproduction remains unexplored, even though many bug reports include images. For instance, RegDroid [17], which is the most comprehensive

dataset of Android functional bugs, features images in 41.35% (165/399) of its bug reports. Similarly, AndroR2 [2], [18], the widely-studied dataset for Android bug reports, contains images in 23.33% (42/180) of its reports. To address this gap, we conducted an empirical study to analyze images in bug reports. Our study is guided by the following primary research questions:

RQ1: How do the number and types of images vary across bug reports? This research question addresses both quantitative (e.g., single-image vs. multiple-image reports) and categorical (e.g., UI screenshots vs. non-UI screenshots) aspects of image usage in bug reports. Understanding these patterns is essential for assessing how images contribute to bug reports. This foundation can inform the design of automated bug reproduction tools by identifying image patterns that are likely to be effective in automation or pinpointing specific types of images with the greatest potential to enhance automation processes.

RQ2: What are the functional roles of images in bug reports? Textual information in bug reports is typically organized into functional categories, such as Steps-to-Reproduce (S2Rs) and Observed Behavior (OB) [19], [20], each emphasizing different key information and requiring distinct strategies to effectively utilize the information for bug reproduction. For instance, textual S2Rs incorporate specific action verbs (e.g., “click”) and nouns related to UI elements (e.g., “menu”), which have influenced the creation of various tools [7], [9], [10]. These tools utilize S2Rs through a phase known as S2R Entity Extraction. During this phase, crucial entities are extracted from S2Rs, identifying important action-noun pairs to facilitate automated bug reproduction. Similarly, when using crash logs, identifying key information like the activity name where the bug occurred and the specific error exception is crucial for accurately reproducing the bug. Thus, this research question aims to examine the functional roles that images play in bug reports, identifying patterns within these roles and analyzing the specific information emphasized for each role. This analysis can inspire future research by providing a foundation for leveraging images more effectively: it clarifies how to differentiate between image roles and, for each role, highlights the essential information that should be prioritized or extracted to assist in bug reproduction.

RQ3: Are images in bug reports documented? While a plain image can convey information, supporting text is essential for a complete understanding, especially when images may serve various functional roles (as explored in RQ2). Correctly identifying these roles for reproduction requires adequate documentation. Therefore, this research question investigates the extent to which images in bug reports are documented. If documentation is present, it examines its form—such as annotations or accompanying text. When documentation is absent, we need to consider alternative ways to interpret and utilize the image effectively to understand its purpose and relevance.

RQ4: What types of images have the most significant impact on the effectiveness of automated bug reproduc-

tion? This research question explores the role of images in automated bug reproduction by distinguishing between image types that contribute positively to the process, those with negligible impact, and those that may even hinder effectiveness. We aim to understand why certain images aid or obstruct automation, and to uncover opportunities to leverage these insights—whether to refine existing tools or to design new ones that make more strategic use of image.

Our study highlights the importance of images in automated bug reproduction and presents several key findings regarding the effective use of different image types in bug reports. In summary, this paper makes the following contributions:

- This study represents the first systematic study on the images within bug reports. It investigates the frequency of image inclusion, the functional roles these images serve, how they are documented, and evaluates their impact on the performance of current automated bug reproduction tools.
- Several key findings offer valuable insights and directions for future research on leveraging images in bug reports to enhance automated bug reproduction and new research direction.
- We have made the replication package available [21].

II. METHODOLOGIES

This section presents our methodology for dataset collection and analysis. An overview of our study is shown in 1.

A. Dataset Construction

We constructed two datasets to address the four research questions. The construction of each dataset follows different methodologies suited to their respective objectives. Dataset₁, designed to answer RQ1 through RQ3, focuses on gathering a wide range of real-world bug reports, including those with images, to analyze their characteristics and variety. This dataset does not require verification of whether the bugs are still reproducible, as its primary purpose is to provide a broad overview of images’ characteristics in bug reports rather than current reproducibility verification. In contrast, Dataset₂, which is designed to answer RQ4, focuses on conducting experiments using existing tools to reproduce the bug reports. Therefore, this dataset requires manual verification of the bugs to ensure they are reproducible

Dataset₁. We adopted established practices [2], [17], [18], [22] for gathering real-world bug reports to ensure our analysis is based on authentic and practical scenarios. We initially collected 645 real-world Android apps from F-Droid [23], a platform known for its extensive collection of open-source Android apps, most of which are hosted on GitHub. Using the GitHub REST API [24], we crawled 257,140 issues from the GitHub repositories of these 645 apps. To identify bug reports, we applied the following filtering criteria: (1) the issue must include at least one label containing the keyword “bug”. (2) to focus exclusively on mobile app-related bug reports, any issue containing the terms “windows”, “linux”, “desktop”, or “tv” (case-insensitive) was excluded from our analysis. As a result,

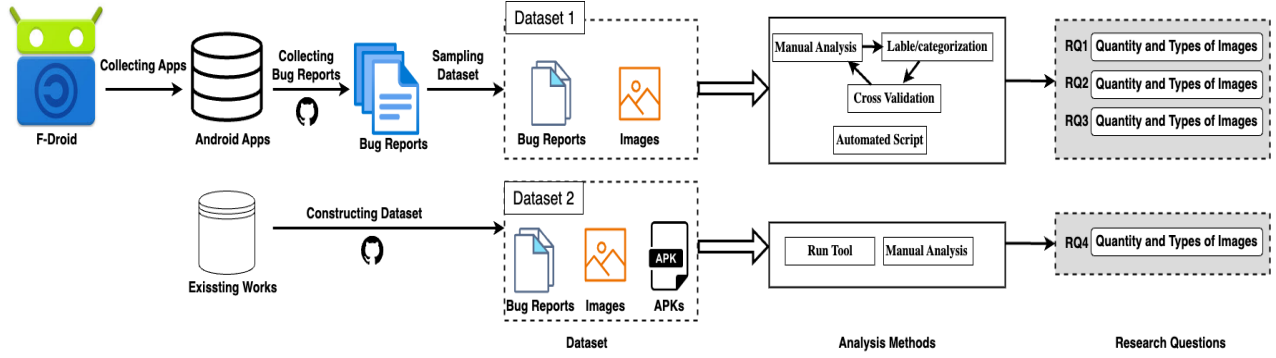


Fig. 1. Overview of our study

we obtained a total of 50,988 Android bug reports, of which 7988 (15.67%) include images. To ensure a representative yet manageable dataset for answering our research questions (RQs), we applied standard statistical sampling techniques with a 95% confidence level and a 5% margin of error [25]. From the total of 7,988 bug reports containing images in our initial pool of 597 apps, we randomly sampled 367 reports. This finalized Dataset₁, which is used for answer RQ1, RQ2, and RQ3.

Dataset₂. We constructed the second dataset to evaluate the performance of existing tools for handling image-containing bug reports (RQ4). To create this dataset, We followed the dataset construction methods used by established bug reproduction tools [8]–[11], selecting bug reports from a well-known dataset to ensure their representativeness. Additionally, apps in this dataset are well-established within the field of Android bug reproduction. By focusing on these well-studied apps in automated bug reproduction, we leveraged existing knowledge and resources to reduce the time required for manually reproducing we extended our collection beyond the available reports by examining GitHub repositories to identify more image-containing bug reports. We applied similar criteria for examining bug reports: (1) they must have accessible APK files, and (2) they must still be reproducible. Additionally, we included one criterion specific to our study: each bug report must contain images, aligning with our focus on image-containing bug reports. As a result, we collected 42 bug reports that include at least one image as Dataset₂ to answer RQ4.

Finding 1: The dataset construction process shows that 15.67% of Android bug reports contain images, emphasizing their frequent inclusion and the need to explore their potential role in automated bug reproduction.

B. Analysis Methods.

To address the research questions, a combination of automated scripts and manual analyses was employed to ensure reliability and validity. To understand the prevalence of images in bug reports (RQ1), an automated script scanned each report to identify and count images, followed by a manual

classification of images as either UI screenshots or other types of static visualizations. To investigate the functional roles of images in bug reports, the authors conducted a multi-round classification process, independently analyzing images to categorize their functions and assess documentation relevance within the bug reports. After each round, they discussed and refined classification categories, adding new ones as needed to reach a consensus. To analyze the impact of images on the effectiveness of existing bug reproduction tools (RQ3), we tested reports containing images using automated reproduction tools, examining their success rates when image data was omitted and identifying limitations in image handling. To validate our findings, we used two third-party datasets: AndroR2 [2], [18] and RegDroid [17] to ensure that our results were consistent and generalizable.

C. Selecting Bug Report Reproduction Tools

Table I summarizes ten recent state-of-the-art studies on automated bug report reproduction, none of which have considered images. We selected tools that can accept free-text descriptions of bug reports as input. Other tools, such as CrashTranslator [13], which relies on logcat, GifDroid [14], which utilizes video input, and Roam [8], which requires JSON-formatted input, are beyond the scope of our study. Our study aims to explore how images within textual descriptions can enhance automated bug reproduction. In contrast, logcat outputs and videos represent separate entities within the bug report structure. Yakusu [6] was excluded due to its limited relevance compared to more recent and advanced tools that have demonstrated superior performance. Additionally, DroidScope [12] was not included because of its closed-source nature. Consequently, we selected four bug report reproduction tools [7], [9]–[11] for our analysis in RQ3.

Specifically, **ReCDroid** [7] is an automated bug reproduction tool that follows a two-phase approach: S2R entity extraction and S2R entity matching. It combines natural language processing (NLP) with heuristic grammar patterns for entity extraction and employs a guided depth-first search to dynamically explore the application, aiming to match S2R entities with GUI elements to eventually reproduce event sequences that trigger the reported bug. ReCDroid was one of the earliest

works in this area and is a popular baseline for evaluating new testing techniques. **ReproBot** [9] also uses NLP but combines it with reinforcement learning (RL) techniques like Q-learning to optimize the search of the UI actions for crash reproduction. The Q-learning algorithm helps ReproBot to account for missing steps and identifies UI actions that overall match the S2Rs. **AdbGPT** [10] is the first work that uses Large Language Models (LLMs) through prompt engineering and chain-of-thought reasoning, enabling accurate S2R extraction and replay without extensive training data. **ReBL** [11] bypasses the traditional two-phase approach and the use of S2R entities. Instead, it leverages the entire textual bug report and employs LLMs to guide the reproduction process in a feedback-driven manner. This approach is more flexible and context-aware than the traditional step-by-step entity matching method. It is the first tool capable of reproducing bugs using entire bug reports without relying on S2R entities.

III. EMPIRICAL STUDY

A. Quantity and Types of Images (RQ1)

This research question examines patterns in the quantity and types of images used in bug reports, offering insights into their distribution and usage. We analyze the average number of images per report, categorize them by type (e.g., UI screenshots vs. other images), and identify how many reports contain single or multiple images. This analysis provides a foundation for further studies to explore the potential of using image usage in bug reports for automated bug production.

In analyzing the 367 bug reports in Dataset₁, 70.77% of reports contained exactly one image, while 29.23% included multiple images, with the number of images per report ranging from 1 to 7. On average, there were 1.48 images per bug report. Of these, 95.18% were UI screenshots, while a smaller portion (4.82%) of these images included pictures of code snippets, crash logs, or photos of the UI screen taken by another device.

Finding 2: 70.77% of reports included one image, while 29.23% had multiple images (ranging from 1 to 7 per report). Of all images, 95.18% were UI screenshots, and 4.82% were non-UI (e.g., code snippets, error logs).

Since the majority of reports contain a single image, it is crucial to understand the functional role of this image in single-image reports. What kind of information does it provide, and could it effectively be leveraged in automated bug reproduction? Conversely, in reports with multiple images, it is important to consider whether these images serve the same functional role or convey different aspects of the bug, as well as to examine the logical sequence between images. With these questions in mind, we will further explore the functional roles of images in RQ2.

B. Functional Roles of Images (RQ2)

Bug reports typically consist of several key components that describe an issue, each serving a specific purpose: Steps to

Reproduce (S2Rs), Observed Behavior (OB), and Expected Behavior (EB) [19], [20]. Following this structure, images in bug reports can be classified into four main categories: *S2R*, *OB*, *EB*, and *Others*. The *Others* category includes images that do not fit into the first three categories (e.g., screenshots of code). Additionally, we found that images in *S2R* category could be further categorized into three distinct types based on their characteristics: *S2R_{standalone}*, *S2R_{context}*, and *S2R_{outcome}*. Just like textual descriptions in bug reports, bug reproduction tools that can correctly classify images into their respective functional roles are essential for successful bug reproduction.

1) ***S2R_{standalone} image***: An *S2R_{standalone} image* is a self-contained visual representation of an S2R that requires no accompanying text. These images are typically annotated to emphasize target user interface (UI) elements and the actions to be performed. For instance, in Fig. 2-(a), the first S2R is depicted entirely through an annotated image: the target element "an.db" is circled in red, and the action "click" is clearly indicated next to it. This effectively communicates the reproduction step "click on an.db." Images that fully capture an entire step-to-reproduce (S2R) without any accompanying text are important for effective bug reproduction.

Neglecting such images could lead to missing steps, a major factor impacting the performance of automated bug reproduction processes.

2) ***S2R_{context} image***: An *S2R_{context} image* complements textual S2R instructions by providing additional visual context. Typically, it accompanies text in a pattern, such as "textual S2R + image". The textual S2R offers the primary guidance for reproducing the step, while the image adds crucial visual cues, forming a complete S2R together. These images often highlight target UI elements or required text inputs (e.g. Figure 2-(b)). Users tend to include images to depict target elements that are difficult to describe, especially when the element is uncommon or lacks a clear label—such as a menu icon or a "more options" button. When images are used to show required text input, it often indicates that the input is complex or specially formatted, not just simple text that can be easily typed in the bug report. For example, in the bug report shown in Figure 2-(b)), the third S2R states, "Click on the tabs display point," followed by an annotated image that indicates which UI element is the "tabs display point," helping the developer quickly and accurately locate the target element.

Such images reduce ambiguity in S2R instructions by providing visual cues of the target, enhancing clarity and understanding. This addresses another main challenge in existing work, where the bug report written by the user might be ambiguous and not clear due to their writing habits and lack of technical background.

3) ***S2R_{outcome} image***: This type of image captures the state of the UI after a S2R has been replayed. Unlike other S2R images that either represent S2Rs or are used to complement S2Rs, *S2R_{outcome} images* focus solely on displaying the results following the execution of the S2R. In manual bug reproduction, an *S2R_{outcome} image* is primarily used to verify

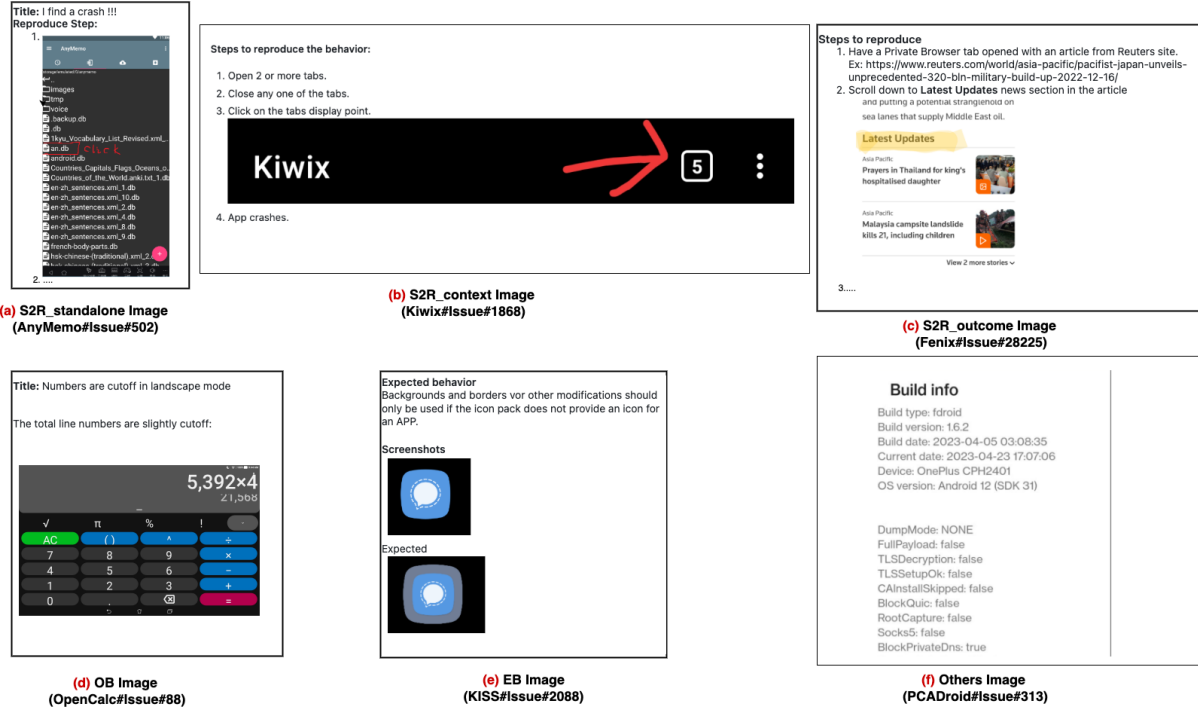


Fig. 2. Examples of Images in Different Functional Roles

whether an S2R has been successfully replayed by comparing the image with the actual UI page. This visual confirmation allows developers to ensure that the action produced the expected outcome, thereby validating the accuracy of the reproduction step.

However, in automated bug reproduction, existing tools typically do not perform any verification of S2R replaying. Instead, these tools rely solely on the presence of the target of the next S2R to advance through the reproduction steps. For example, given two steps, S_1 and S_2 , after performing S_1 , the tool searches for the target element of the subsequent S2R, S_2 , within the current UI state. If the element for S_2 is not found, the tool may backtrack to a previous UI state to attempt replaying S_1 again. If the target element is found, the tool proceeds to replay S_2 and continues with the subsequent steps. This approach assumes that the mere presence of next target elements is sufficient to determine the success of each step. Consequently, the utility and necessity of $S2R_{outcome}$ images in automated bug reproduction remain unexplored. While these images undeniably offer valuable visual confirmation in manual processes, their integration into automated systems has not been considered. Determining whether incorporating $S2R_{outcome}$ images would enhance automated bug reproduction requires experimental validation and further investigation.

Finding 3: S2R images fall into three subcategories—context, standalone, and outcome—based on how they convey information. $S2R_{context}$ and $S2R_{standalone}$ images reduce ambiguity by clarifying reproduction steps and providing visual cues to support textual guidance. $S2R_{outcome}$ images provide essential visual confirmation in bug reproduction but are currently overlooked by existing tools.

4) **OB image:** OB image visually illustrates the reported bug symptom, validating the existence of the bug. This saves developers from spending time questioning the existence of the bug when they are unable to reproduce it. From the perspective of bug reproduction, OB images serve dual purposes: visually illustrating bug symptoms and aiding the step replay. OB images clearly convey the bug symptoms, helping developers comprehend the issue and recognize when the bug is triggered during manual reproduction. Moreover, OB images provide additional context and visual cues to guide navigation to the target page where bugs occur.

For example, the images in Figure 2-(d) shows OB images, illustrating the non-crash bug symptoms. In Figure 2-(d), the bug report shows the OB image with short description but not S2Rs for reproducing the bug. Nevertheless, the image provides useful context: aside from displaying the overlap bug symptom, the image reveals that this issue occurred on the calculator page. The digits shown on the screen imply

that to replicate the overlap, it might be necessary to input an equation in the calculator. Thus, beyond highlighting the bug symptom, *OB images* offers clues about the specific page where the overlap took place.

To be clear, although *OB images* and *S2R_{outcome} images* share similarities, as both capture the UI results of specific steps in the reproduction process, the key distinction lies in whether the image displays the reported bug symptom. An *OB image* is essentially a subset of *S2R_{outcome} images*, specifically capturing the result of the final S2R where the bug symptom is visible. In contrast, *S2R_{outcome} images* may depict the result of any step along the reproduction process and do not show the bug symptom. Therefore, if an image is an *S2R_{result} images* of the last S2R, it is classified as an *OB images* rather than an *S2R_{outcome} images*.

5) **EB image:** An *EB image* provides a visual reference of the expected or correct behavior, often captured from a previous version of the application. In our dataset, we found that bugs related to issues like padding, text alignment, and color settings (including dark and day modes) frequently include EB images to help developers visually compare and identify the discrepancies. Additionally, EB images often appear alongside OB images for direct comparison, as illustrated in Figure 2-(e).

Despite this benefit, existing automated tools ignore not only EB images but also any textual information related to EB. This is because EB information serves as a reference for how the application should behave, rather than explicitly showing a failure. This implicit nature makes it challenging for automated tools to utilize, unlike S2Rs (which provide direct instructions for reproducing the bug) or OB images (which display the exact symptoms of the issue). However, we should not overlook the potential of leveraging EB images to enhance automated bug reproduction. One possible integration is to use EB images in conjunction with OB images for automated oracle checking. By comparing EB images (representing correct behavior) with OB images (displaying bug symptoms) and the actual UI page, automated tools can more effectively identify discrepancies. Another approach is to view the EB image as the inverse of the OB image, suggesting that the EB image also describes the same UI page. This perspective allows EB images to implicitly provide information about the UI page (e.g., the target state) where the bug occurs.

6) **Others image:** These images can be classified as follows: (i) non-UI images, such as screenshots of source code or error logs; (ii) combined screenshots, which are single images created by merging multiple screenshots, making them challenging to interpret due to the diverse information they contain and the lack of clear context, and the difficulty of separating them; and (iii) UI images that are not directly related to OB, EB, or S2R, such as a screenshot of the settings screen that simply shows the application's configuration (e.g., Figure 2-(f)). Although not directly illustrating the bug, these images (e.g., logs) can be valuable by revealing settings or conditions impacting the application's behavior.

Table II shows distribution of image roles across image bug reports. The percentages refer to the proportion of bug reports

TABLE II
DISTRIBUTION OF IMAGE ROLES ACROSS SINGLE-IMAGE AND MULTIPLE-IMAGE BUG REPORTS

Image Roles	Sgl-Img BR	Multi-Img BR	Overall
<i>S2R_{standalone}</i>	0	2.60%	0.82%
<i>S2R_{context}</i>	1.79%	20.78%	7.76%
<i>S2R_{outcome}</i>	1.19%	10.39%	4.08%
<i>OB</i>	93.45%	90.91%	92.65%
<i>EB</i>	2.38%	46.75%	16.33%
<i>Others</i>	1.19%	20.78%	7.35%

containing images in each functional role, rather than the percentage of each image type over the total number of images. In the Overall column, the majority of bug reports (92.65%) contain at least one image serving the OB role, followed by EB images and S2R images. This distribution emphasizes the popularity of using images to visually present observed and expected outcomes, likely due to the variety of non-crash symptoms and expected behaviors that are more effectively conveyed through visuals rather than textual descriptions. Images provide a clearer representation of the issues and the desired results, especially in cases where complex visual elements are involved, making them a valuable resource in the bug reporting and reproduction process.

We further examine the functional roles of images in bug reports by differentiating between those that contain a single image and those with multiple images. We hypothesize that this separation will aid in classifying images into their respective functional roles more effectively. For instance, if most bug reports with a single image predominantly feature OB images, it would be simpler to categorize these images accordingly. As the result, the last two columns of Table II suggest the following:

First, *OB* are dominant in both single-image and multiple-image bug reports, with 93.45% of single-image reports and 90.91% of multiple-image reports containing at least one OB image. This highlights the popularity of using images to illustrate observed behavior, underscoring their value in effectively communicating the symptoms of a bug. Additionally, we can infer that when a bug report includes only one image, it is most likely to be an *OB* image, suggesting that capturing the observed behavior is often the primary purpose of single-image bug reports.

Second, apart from *OB* images, the proportion of all other roles increases significantly in multiple-image reports. This can be attributed to two main reasons: (1) when a bug report includes more than one image, there is greater diversity in the roles these images can fulfill, and (2) in multiple-image bug reports, each image can serve a different role, leading to overlapping classifications. Consequently, the cumulative percentage for each role exceeds 100% as individual reports can contain multiple images fulfilling multiple functional roles. This highlights how multiple-image bug reports are often more comprehensive, capturing various aspects of the bug, such as observed behavior, expected behavior, and contextual information, which collectively enhance the clarity and reproducibility

of the bug report.

Finding 4: First, the majority of bug reports (92.65%) contain at least one OB image. This trend is especially strong in single-image reports, where 93.45% are OB images, suggesting that when only one image is included, it is most likely to represent the observed issue. Second, multiple-image reports contain diverse images serving different roles, which implies that if future work aims to leverage these images to facilitate automated bug reproduction, it will be crucial to understand how to identify and differentiate these roles effectively.

C. Documentation of Images (RQ3)

During the classification of the functional roles of images (RQ2), it is evident that static images provide limited information on their own. To accurately differentiate the roles of these images, it is necessary to consider the accompanying textual information in the bug report. This additional context is important for interpreting the intent and relevance of the images. Therefore, documentation becomes a key aspect. We studied the documentation of images—specifically, those that include annotations or are accompanied by explanatory text. Such documentation helps indicate the purpose or context of the image, enhancing its utility in understanding and reproducing the bug. An image was considered documented following this criteria: we first determined whether an image contained Explanatory Text. If it did, we classified it under this category. If an image lacked explanatory text, we then assessed whether it was placed under a section with descriptive subsection titles. If an image met neither of these criteria, it was categorized as a plain image.

- **Explanatory Text (38.06%).** Descriptive text accompanying the image that explains what the image shows or how it relates to the issues being reported.
- **Descriptive Subsection Titles (5.17%).** Clear and informative headings above the image that indicate what the image is illustrating. Titles like "Observed Behavior" are helpful, whereas generic titles like "Screenshot" do not provide meaningful context. (e.g., plain images but placed under a subsection with a descriptive title)

Of the images in bug reports, 43.23% were identified as documented images, while 56.77% were categorized as plain images. Plain images typically occur in two scenarios: (1) multiple consecutive images are placed together without accompanying text to clarify their differences or relationships, and (2) many bug reports use a template with sections labeled "Screenshots" or "Other." Although these sections are well-intentioned, users often upload screenshots without additional explanation, making it difficult to interpret the images and challenging to identify which parts of the text they are intended to support.

Finding 5: Documented images (43.23%) with explanatory text or descriptive titles enhance clarity, while plain images (56.77%). If one intends to use images in automated bug reproduction, this lack of clear context complicates the classification process, making it challenging to accurately interpret each image's functional role.

D. Existing Tools (RQ4)

In this section, we assess how images in bug reports influence the effectiveness of automated bug reproduction through two experiments. First, we examine whether existing tools, which currently lack the capability to process images as input, can successfully reproduce bugs when images are omitted from the reports. Second, we explore whether the functional content of images, if ideally translated into descriptive text, can enhance the accuracy and effectiveness of bug reproduction with existing tools.

1) *Experiment Design and Setup:* The first experiment uses textual information from bug reports as input to determine whether existing tools could automatically reproduce bug reports. If the tools successfully reproduced the bugs within a reasonable timeframe without utilizing the images from the bug reports, it would suggest that the images may not be critical, as the textual hints provided are sufficient to guide the existing tools. The second experiment focused on the cases where the first experiment failed. In these cases, we manually turned the information from the images into text descriptions. We divided the images into two groups: S2R and non-S2R. S2R Images provide clues about specific actions, targets, or elements involved in the reproduction steps. We translated these images into precise action-target tuples that existing tools can interpret. In contrast, non-S2R images, which lack specific actions and targets and cannot be directly converted into action-target tuples, were handled by listing the names of the UI elements on the images. This included details such as the page title (if any) and the various UI elements present in the image. If supplementing the translated information leads to successfully reproducing previously failed cases, it suggests that precisely capturing the image's intent and extracting key information can effectively leverage images to enhance bug reproduction.

We conducted our experiment on a physical x86 machine running Ubuntu 16.04, equipped with an i7-4790 CPU @ 3.60GHz and 32 GB of memory. We followed the same evaluation settings as in ReCDroid [7], ReproBot [9] AdbGPT [10], and ReBL [11] limiting each experiment to a maximum runtime of one hour. If an experiment exceeds this time limit, it is terminated and marked as a failure. For ReBL [11], which includes a summarization mechanism, we maintained the same experimental settings, setting the summarization threshold to three iterations.

2) *Results:* Table III shows detailed results of the bug reproduction for each bug report by the state-of-the-art tools. 23.8% of bug reports were successfully reproduced by at least

TABLE III
EXPERIMENTAL RESULTS FOR RQ4.

ID	Bug Report	RD	RB	AG	BL	ID	Bug Report	RD	RB	AG	BL
1	AFM-1657	×	×	×	×	22	AFM-1794	✓	✓	✓	✓
2	AFM-1795	✓	✓	✓	✓	23	AndOTP-827	×	×	×	×
3	AFM-1796	×	×	✓	✓	24	AndOBD-243	✓	✓	✓	✓
4	AFM-2477	×	×	×	×	25	Android-1248	×	×	×	×
5	andOTP-551	×	×	×	×	26	AnyMemo-500	×	×	×	×
6	andOTP-827	×	×	×	×	27	Fenix-27725	×	×	×	×
7	Anki-13919	×	×	×	×	28	Fenix-27987	×	×	×	×
8	Anki-14609	×	×	×	×	29	Fenix-28086	×	×	×	×
9	Anki-16325	×	✓	×	×	30	k9-4804	×	×	×	×
10	AnkiDroid-6228	×	×	×	×	31	Markor-1815	×	×	✓	✓
11	AnkiDroid-7286	×	×	×	×	32	Markor-1961	×	×	×	×
12	AnkiDroid-9005	✓	✓	✓	✓	33	NewPipe-10090	×	×	×	×
13	AnyMemo-502	×	○	○	○	34	NewPipe-10380	×	×	×	×
14	Kiwix-1868	×	×	×	×	35	NewPipe-10646	×	×	×	×
15	Markor-1020	×	×	×	✓	36	NewPipe-41	×	×	×	×
16	Markor-1565	×	×	×	×	37	NewPipe-56	×	×	×	×
17	Markor-1729	×	×	×	×	38	News-156	✓	✓	✓	✓
18	Markor-550	×	×	×	×	39	OpenNoteScanner-166	×	×	×	×
19	Markor-567	×	×	×	×	40	OpenNoteScanner-39	×	×	×	×
20	OmniNotes-634	×	×	○	○	41	Aopentasks-898	✓	✓	✓	✓
21	Trickytripper-49	×	✓	✓	✓	42	PdfViewer-55	×	×	×	×

¹ ✓ Reproduction succeeds without considering the image.

² ○ Reproduction succeeds only after manually providing the information of the image.

³ × Reproduction failed in both cases.

one tool, while 76.2% were not reproduced by any existing tools. We further analyze the failed cases and summarize the following key observations.

- ***S2R_{context} image and S2R_{standalone} image play a critical role in filling in missing steps during the bug reproduction process. (Example: AnyMemo#502).*** As introduced in RQ2 of our empirical study (Section 2) S2R_{standalone} represents a complete Step-to-Reproduce (S2R) image that visually illustrates specific actions, while an S2R_{context} image complements the textual S2R by providing additional context. Our experimental results validate these findings, showing that omitting these images often leads to incomplete information or overlooked steps, resulting in failed reproduction attempts. Additionally, in a second experiment, we attempted to convert the visual information from images into action-target components to enhance the textual reports. This approach led to successful reproduction in two previously failed cases, demonstrating the potential of leveraging S2R images to complete the S2Rs. This proof-of-concept suggests that developing methods to translate S2R images into actionable text could help address these challenges and harness the benefits of S2R images in future automated bug reproduction.
- ***Images in bug reports are less critical if the textual S2Rs are sufficient. (Example: Trickytripper#49).*** Sufficient information refers to Steps to Reproduce (S2Rs) that are comprehensive and detailed, providing all the necessary target-action information required for accurately reproducing the bug. In these cases, images become less critical in automated bug reproduction, as existing tools have made significant progress using S2Rs alone, applying various techniques

(e.g., reinforcement learning, and prompt engineering) and achieving promising results. This raises an interesting question: can we assess if textual information is sufficient for bug reproduction before running the tool? If so, we could selectively use text-only tools or incorporate images as needed. Beyond this, the classifier could help determine whether to leverage LLM-based [10], [11] or NLP-based [7] tools, depending on whether the textual information is structured and the action-target details are clearly documented, opt for tool using crash log as input if a crash log is available [13], or apply video-based reproduction tools [14] when videos are provided. Therefore, it might be beneficial to build a classifier to assess information sufficiency, enabling more efficient tool selection and improved performance.

- ***The role of non-S2R images needs to be further determined.*** S2R images can be accurately translated manually into a textual format, such as action-target pairs, which existing tools can readily use. This structured format is effective because it aligns well with the current capabilities of automated tools. However, when images convey information beyond simple action-target pairs—such as OB (Observed Behavior) images that illustrate the varying symptoms or EB (Expected Behavior) images that depict the desired outcome—it becomes challenging to convert them accurately into text, and text alone cannot intuitively capture these nuances. A significant challenge in existing works is the bug oracle, which requires a comprehensive understanding of OB information. Therefore, exploring OB and EB images might be a breakthrough in addressing this challenge, as incorporating images can supplement and enrich the textual OB and EB information.

Finding 6: First, *S2R_{context}* and *S2R_{standalone}* images are essential for automated bug reproduction and can be effectively translated into textual tuples, aligning well with the current capabilities of automated tools. Second, non-S2R images are more complex because they lack the clear action-target patterns found in S2R images, making them harder to interpret. These images often convey nuanced information, such as observed or expected behaviors, which require further investigation to effectively integrate into automated bug reproduction.

IV. REPRESENTATIVENESS OF STUDY

In this section, we validated the representativeness of our study for RQ2 to ensure that our findings are not limited to the specific characteristics of the initial sample. We performed this validation for RQ2 specifically because it involves classifying images into six distinct roles, a more nuanced task than RQ1 and RQ3, which focus on characteristics like quantity and documentation. We validated our results by applying the same analysis to two independent third-party datasets AndroR2 [2], [18] and RegDroid [17]. The two datasets were not constructed by the authors of this paper, and the presence of images in bug reports was not one of the criteria in their construction, nor were images in bug reports studied in subsequent analyses.

AndroR2 contains 23.33% (42/180) of bug reports with images, though three of these reports either no longer have access to the images or the images are externally hosted. For RegDroid, 41.35% (165/399) of bug reports include images, but five of these reports lack access to the images. We focused on bug reports with accessible images, which include 36 from AndroR2 and 160 from RegDroid. We applied the same methodologies as used in RQ2 to classify the functional role of images in the bug reports. The results were then compared to our own dataset to determine if the findings from these third-party datasets align with ours. This analysis helps assess the representativeness of our findings.

TABLE IV
DISTRIBUTION OF IMAGE ROLES ACROSS SINGLE-IMAGE AND
MULTIPLE-IMAGE BUG REPORTS

	AndroR2	RegDroid	Our Dataset
<i>S2R_{standalone}</i>	0	0.6%	0.8%
<i>S2R_{context}</i>	9.8%	11.1%	7.8%
<i>S2R_{outcome}</i>	2.9%	8.3%	4.08%
<i>OB</i>	93.45%	100%	92.65%
<i>EB</i>	18%	5.6%	16.33%
<i>Others</i>	6.5%	5.6%	7.35%

Table IV presents the results of classifying image roles of our dataset and the two third-party datasets. The findings reveal a consistent trend: most bug reports feature OB images, following EB and S2R images. The similar distribution of image roles across all three datasets indicates that our study's findings are generalizable beyond the randomly sampled dataset. By classifying and evaluating images in AndroR2

and RegDroid, we demonstrate that the identified patterns are consistent across different data sources and contexts. This cross-validation confirms the robustness of our findings regarding the utilization of images in bug reports, thereby enhancing the validity and reliability of our conclusions.

V. IMPLICATIONS AND OPPORTUNITIES

In this section, we explore the implications of our study, emphasizing the contributions of images in bug reports and how developers and researchers can leverage them to improve automated bug reproduction.

A. Bug Reproduction

Implication 1: Understanding the various types of images and identifying their roles is important. RQ2 findings indicate that different images serve distinct functional roles, each providing unique information, while RQ3 demonstrates that images in multi-image bug reports fulfill varied purposes. This complexity makes it difficult to balance the information presented in both text and images within a bug report. How can we effectively identify the roles of these images and understand their relationship to the accompanying text?

Opportunities: A human-like tool could employ a multi-agent system, where each agent is responsible for handling textual information and images according to their distinct functional roles. This collaborative approach enables agents to independently analyze the different roles of images and text before combining their findings to make decisions for accurate bug reproduction.

Implication 2: Images can complement S2Rs to minimize the risk of missing steps. When considering images in bug reports for automating bug reproduction, it is essential to at least consider S2R images, just as most existing tools focus on the textual S2R for bug reproduction while overlooking other textual information. S2R images are critical for step replay, especially for *S2R_{context}* images and *S2R_{standalone}* images (Finding 6). The context images provide necessary target information or input details, while standalone images represent complete steps on their own. Without these standalone images, it's like creating a missing step, which increases the challenge of bug reproduction. Although some existing tools can bridge missing steps, incorporating images can improve the efficiency of the process.

Opportunities: To effectively utilize images in bridging the gaps of missing steps in automated bug report reproduction, we can apply Optical Character Recognition (OCR) techniques [26] and heuristic patterns to extract key information from S2R images, such as target elements and actions. In addition, we can implement a multimodal approach with reasoning capabilities that generate suggestions based on the available textual S2R and the corresponding S2R image.

Implication 3: Images could be a breakthrough in the development of automated reproduction for non-crash bugs. Non-crash bug reproduction presents a significant gap in this field, particularly due to the diverse symptoms associated with non-crash bugs [11], [17], [27], [28]. There are many oracle

techniques, each targeted at certain types of bug. [27]–[35]. Existing bug reproduction works predominantly targets crash bugs, largely because of the lack of effective verification capabilities for non-crash bug reports, which makes them more difficult to diagnose and reproduce. Recent studies have only begun to explore the potential of large language models (LLMs) for handling non-crash bug reports reproduction, yet they have not conducted detailed evaluations or analyses. As a result, there is an urgent need for tools that can reproduce non-crash bug reports, as well as more general tools for bug symptom verification. Utilizing images could be instrumental in addressing this challenge, especially when the focus shifts to verification rather than merely replaying steps. Our study has shown that many images in bug reports capture non-crash observations and expected behavior, providing valuable visual insights (Findings 4). By analyzing these non-crash images, we can gain a deeper understanding of non-crash behaviors, enabling us to detect non-crash symptoms in the user interface (UI) more effectively, rather than solely identifying non-crash states within the UI.

Opportunities: To automatically reproduce non-crash bug reports, we can leverage multimodal learning to capture non-crash symptoms. This involves exploring whether a pre-trained model can accurately detect bug symptoms and compare the user interface (UI) with the Observed Behavior (OB) image through prompt engineering, or if fine-tuning the model is necessary to improve performance.

Implication 4: Images should be placed in their appropriate sections. RQ3 investigates the documentation of images and highlights a common issue: many images are presented without sufficient explanatory context. Plain images that lack accompanying text or are placed under generic, non-descriptive subsection titles create challenges for users, who must sift through the entire bug report to discern the image’s purpose and its relationship to specific text. For instance, a plain image in a generic “Screenshot” section requires readers to infer whether it pertains to Steps to Reproduce (S2Rs), Observed Behavior (OB), or is unrelated. This ambiguity makes it difficult to understand the image’s functional role and purpose in the bug report, ultimately hindering its effective use.

Opportunities: Methods can be developed to help users place images close to relevant text or sections of the bug report, facilitating a clearer understanding of the visual content in relation to the issue. It may not be a good idea to create a standard template for additional or screenshot sections; however, if a screenshot section is necessary, users should be encouraged to provide explanations for each image. Simple statements like “This is what I observe” or “This relates to Step 1” can offer valuable context. This additional information will assist automated techniques, such as machine learning, in determining the appropriate role of each image.

VI. THREATS TO VALIDITY

Our study may suffer from some threats to validity. First, although we used a 95% confidence level, some unique or less common characteristics of bug reports may still be

underrepresented, potentially impacting our findings. Second, the relatively small number of bug reports in RQ4 may limit the generalizability of our conclusions. This limitation is further reflected in the size of the S2R dataset, which, while imbalanced, mirrors real-world scenarios. The limited number of bug reports containing S2R may impact the generalization of our findings, similar to other empirical studies. However, as this is the first dataset of image-based bug reports, we plan to expand its size as part of our future work. Third, the manual classification of image roles introduces potential for human error, even with cross-validation, which may impact labeling consistency. To enhance generalizability, we analyzed two third-party datasets, AndroR2 and RegDroid, to classify the functional roles of images within these datasets. The results from both datasets are consistent with our findings. Fourth, we evaluated two recent tools, AdbGPT and ReBL, in RQ4. Both tools use GPT [36] models, which are continually updated, meaning that the results may not remain consistent over time. Additionally, our findings highlight a critical limitation of current tools: they are primarily designed to process textual S2R information, making them less effective at leveraging the rich information within non-S2R images to improve bug reproduction.

VII. RELATED WORK

Automated Bug Report Reproduction. There are existing approaches that specifically target automatically reproducing Android bug reports as listed in Table I in Section I, including Yakusu [6], ReCDroid [7], GIFdroid [14], DroidScope [12], ReproBot [9], AdbGPT [10], CrashTranslator [13], Roam [8] and ReBL [11]. Section II-C provides introductions to each of these approach. These tools aim to use various types of information in bug reports—such as crash logs, full textual reports, and videos—to automate the bug reproduction process.

However, none have considered the role of images, though GIFdroid automates bug replay from GIFs and videos in bug reports. While it involves processing GIFs and videos using image techniques, its goal is fundamentally different from our study. GIFdroid focuses on replaying sequential interactions from video-based bug reports, leveraging the temporal continuity of frames. In contrast, our work investigates how standalone images—without sequential context—can be utilized for automated bug reproduction. This distinction highlights the unique challenges and opportunities in leveraging images independently, motivating the need for our empirical study.

Bug Report Study and Analysis. There have been several research efforts dedicated to studying and analyzing Android bug reports. For instance, Wendland et al [18] studied 90 manually reproduced bug reports to support automated research in bug analysis and reproduction. Building on this basis, Johnson et al [2] extended the dataset and conducted an empirical study on 180 Android bug reports to examine reproduction challenges and the quality of reported details. Chaparro et al. [20] conducted a study on user-reported behaviors, reproduction steps, and expected outcomes, identifying typical discourse patterns. They later developed Euler [19], an

automated method to evaluate the quality of S2R in Android bug reports based on simple grammar patterns. Liu et al. [37] introduced Maca, a machine-learning classifier that organizes S2R action words into standard categories.

Several studies have explored using LLMs to analyze and interpret bug reports. Lee et al. [38] applied LLMs for bug triage, while Messaoud et al. [39] used a BERT model to detect duplicate reports. Kang et al. [40] proposed generating JUnit test methods for Java programs from bug reports

Some research focuses on improving the bug reporting process. Moran et al. [3] developed Fusion, which uses dynamic analysis to capture app UI events for more informative reports. Fazzini et al. [41] support reporters in writing precise reproduction steps by leveraging static and dynamic analysis to predict next actions. Yang et al. [42] offer a guided system with instant feedback and graphical hints to enhance report quality.

However, these studies focus solely on improving the accuracy of identifying textual S2Rs, with no consideration given to images. Our study examines not only the functional role of images in automated bug report reproduction but also their characteristics within bug reports, offering insights into potential opportunities to leverage them in automated bug reproduction.

VIII. CONCLUSIONS

In this paper, we conduct an empirical study on images in bug reports to reveal their characteristics and functions, analyzing their potential for use in automated bug report reproduction. We identify patterns in image quantity and types, examining their functional roles, usage patterns, and the essential information conveyed by each role. These insights lay the groundwork for future research aimed at differentiating images based on their roles and exploring potential techniques to leverage each role effectively in automated bug reproduction. This research also opens opportunities for enhancing existing tools or developing new, versatile tools that can integrate a broader range of information from bug reports to improve reproduction accuracy.

ACKNOWLEDGMENTS

This work was supported in part by the U.S. National Science Foundation (NSF) under grants CCF-2402103, CCF-2403617, CCF-2403747, and CCF-2211454.

REFERENCES

- [1] "APPLAUSE," <https://www.applause.com/blog/app-abandonment-bug-testing>, 2020.
- [2] J. Johnson, J. Mahmud, T. Wendland, K. Moran, J. Rubin, and M. Fazzini, "An empirical investigation into the reproduction of bug reports for android apps," in *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2022, pp. 321–322.
- [3] K. Moran, M. Linares-Vásquez, C. Bernal-Cárdenas, and D. Poshyanyk, "Auto-completing bug reports for android applications," in *Proceedings of the 2015 10th joint meeting on foundations of software engineering*, 2015, pp. 673–686.
- [4] N. Bettenburg, S. Just, A. Schröter, C. Weiss, R. Premraj, and T. Zimmermann, "What makes a good bug report?" in *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of software engineering*, 2008, pp. 308–318.
- [5] V. Ambriola and V. Gervasi, "Processing natural language requirements," in *Proceedings of the International Conference Automated Software Engineering*, 1997, pp. 36–46.
- [6] M. Fazzini, M. Prammer, M. d'Amorim, and A. Orso, "Automatically translating bug reports into test cases for mobile apps," in *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2018, pp. 141–152.
- [7] Y. Zhao, T. Yu, T. Su, Y. Liu, W. Zheng, J. Zhang, and W. G. Halfond, "Recdroid: automatically reproducing android application crashes from bug reports," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 128–139.
- [8] Z. Zhang, F. M. Tawsif, K. Ryu, T. Yu, and W. G. Halfond, "Mobile bug report reproduction via global search on the app ui model," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 2656–2676, 2024.
- [9] Z. Zhang, R. Winn, Y. Zhao, T. Yu, and W. G. Halfond, "Automatically reproducing android bug reports using natural language processing and reinforcement learning," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 411–422.
- [10] S. Feng and C. Chen, "Prompting is all you need: Automated android bug replay with large language models," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.
- [11] D. Wang, Y. Zhao, S. Feng, Z. Zhang, W. G. Halfond, C. Chen, X. Sun, J. Shi, and T. Yu, "Feedback-driven automated whole bug report reproduction for android apps," *arXiv preprint arXiv:2407.05165*, 2024.
- [12] Y. Huang, J. Wang, Z. Liu, S. Wang, C. Chen, M. Li, and Q. Wang, "Context-aware bug reproduction for mobile apps," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2336–2348.
- [13] Y. Huang, J. Wang, Z. Liu, Y. Wang, S. Wang, C. Chen, Y. Hu, and Q. Wang, "Crashtanslator: Automatically reproducing mobile application crashes directly from stack trace," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.
- [14] S. Feng and C. Chen, "Gifdroid: automated replay of visual bug reports for android apps," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1045–1057.
- [15] S. Feng, M. Xie, Y. Xue, and C. Chen, "Read it, don't watch it: Captioning bug recordings automatically," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2349–2361.
- [16] S. Feng and C. Chen, "Gifdroid: an automated light-weight tool for replaying visual bug reports," in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, 2022, pp. 95–99.
- [17] Y. Xiong, M. Xu, T. Su, J. Sun, J. Wang, H. Wen, G. Pu, J. He, and Z. Su, "An empirical study of functional bugs in android apps," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 1319–1331.
- [18] T. Wendland, J. Sun, J. Mahmud, S. H. Mansur, S. Huang, K. Moran, J. Rubin, and M. Fazzini, "Andror2: A dataset of manually-reproduced bug reports for android apps," in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 2021, pp. 600–604.
- [19] O. Chaparro, C. Bernal-Cárdenas, J. Lu, K. Moran, A. Marcus, M. Di Penta, D. Poshyanyk, and V. Ng, "Assessing the quality of the steps to reproduce in bug reports," in *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, 2019, pp. 86–96.
- [20] O. Chaparro, J. Lu, F. Zampetti, L. Moreno, M. Di Penta, A. Marcus, G. Bavota, and V. Ng, "Detecting missing information in bug descriptions," in *Proceedings of the Joint Meeting on Foundations of Software Engineering*, 2017, pp. 396–407.
- [21] "Replication package," <https://github.com/anonymous10626/replication>, 2024.
- [22] D. Wang, Y. Zhao, L. Xiao, and T. Yu, "An empirical study of regression testing for android apps in continuous integration environment," in 2023

- [23] “F-droid - free and open source software,” <https://f-droid.org/en/>, 2010.
- [24] “Github rest api documentation,” <https://docs.github.com/en/rest>, 2022.
- [25] B. Illowsky and S. L. Dean, *Introductory Statistics*. Houston, Texas: OpenStax College, Rice University, 2013.
- [26] R. Mittal and A. Garg, “Text extraction using ocr: a systematic review,” in *2020 second international conference on inventive research in computing applications (ICIRCA)*. IEEE, 2020, pp. 357–362.
- [27] J. Wang, Y. Jiang, T. Su, S. Li, C. Xu, J. Lu, and Z. Su, “Detecting non-crashing functional bugs in android apps via deep-state differential analysis,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 434–446.
- [28] K. Baral, J. Johnson, J. Mahmud, S. Salma, M. Fazzini, J. Rubin, J. Offutt, and K. Moran, “Automating gui-based test oracles for mobile apps,” in *Proceedings of the 21st International Conference on Mining Software Repositories*, 2024, pp. 309–321.
- [29] J. W. Y. H. J. H. Zhe Liu, Chunyang Chen and Q. Wang, “Owl eyes: Spotting ui display issues via visual understanding,” in *35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2020, pp. 398–409.
- [30] W. Guo, Z. Dong, L. Shen, W. Tian, T. Su, and X. Peng, “ifixdataloss: a tool for detecting and fixing data loss issues in android apps,” in *Proc. of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 785–788.
- [31] J. Sun, “Setdroid: detecting user-configurable setting issues of android apps via metamorphic fuzzing,” in *Proc. of the 43rd International Conference on Software Engineering: Companion Proceedings*. IEEE, 2021, pp. 108–110.
- [32] T. Su, Y. Yan, J. Wang, J. Sun, Y. Xiong, G. Pu, K. Wang, and Z. Su, “Fully automated functional fuzzing of android apps for detecting non-crashing logic bugs,” *Proceedings of the ACM on Programming Languages*, vol. 5, no. OOPSLA, pp. 1–31, 2021.
- [33] C. Escobar-Velásquez, M. Osorio-Riaño, J. Dominguez-Ororio, M. Arevalo, and M. Linares-Vásquez, “An empirical study of i18n collateral changes and bugs in guis of android apps,” in *Proc. of 2020 IEEE international conference on software maintenance and evolution (ICSME)*. IEEE, 2020, pp. 581–592.
- [34] M. Fazzini and A. Orso, “Automated cross-platform inconsistency detection for mobile apps,” in *Proc. of 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2017, pp. 308–318.
- [35] B. Ju, J. Yang, T. Yu, T. Abdullayev, Y. Wu, D. Wang, and Y. Zhao, “A study of using multimodal llms for non-crash functional bug detection in android apps,” *arXiv preprint arXiv:2407.19053*, 2024.
- [36] “Models -openai api,” <https://platform.openai.com/docs/guides/gpt>, 2023.
- [37] H. Liu, M. Shen, J. Jin, and Y. Jiang, “Automated classification of actions in bug reports of mobile apps,” in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020, pp. 128–140.
- [38] J. Lee, K. Han, and H. Yu, “A light bug triage framework for applying large pre-trained language model,” in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–11.
- [39] M. B. Messaoud, A. Miladi, I. Jenhani, M. W. Mkaouer, and L. Ghadhab, “Duplicate bug report detection using an attention-based neural language model,” *IEEE Transactions on Reliability*, 2022.
- [40] S. Kang, J. Yoon, and S. Yoo, “Large language models are few-shot testers: Exploring llm-based general bug reproduction,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2312–2323.
- [41] M. Fazzini, K. Moran, C. Bernal-Cardenas, T. Wendland, A. Orso, and D. Poshyanyk, “Enhancing mobile app bug reporting via real-time understanding of reproduction steps,” *IEEE Transactions on Software Engineering*, vol. 49, no. 3, pp. 1246–1272, 2022.
- [42] Y. Song, J. Mahmud, Y. Zhou, O. Chaparro, K. Moran, A. Marcus, and D. Poshyanyk, “Toward interactive bug reporting for (android app) end-users,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 344–356.